Improved Frobenius FFT for Code-Based Cryptography on Cortex-M4

Myeonghoon Lee, Jihoon Jang, Suhri Kim, and Seokhie Hong

***Abstract*—Polynomial multiplication over finite fields is one of the most significant operations in code-based cryptography, including hamming quasi-cyclic (HQC), which has been selected as a standardized algorithm in the NIST post-quantum cryptography (PQC) round 4 process. In the standardization process, the performance of an algorithm is important not only in general-purpose (GP) systems but also in embedded systems. In particular, NIST has recommended the ARM Cortex-M4 as the benchmark platform for embedded systems. In CHES2021, Chen et al. optimized bit flipping key encapsulation (BIKE) on the ARM Cortex-M4, using the Frobenius Additive FFT as the polynomial multiplication algorithm. However, although HQC was finally selected as a standard algorithm in March 2025, an efficient implementation for the ARM Cortex-M4 platform, which NIST recommends as the benchmark for embedded systems, has not yet been reported. In this article, we propose an optimized implementation of the Frobenius Additive FFT to accelerate polynomial multiplication in BIKE and HQC on the ARM Cortex-M4 platform. Our approach exploits the fact that one operand of field multiplications in the Frobenius Additive FFT is fixed, allowing the transformation of these operations into binary matrix-vector products. We then apply XOR-efficient linear layer techniques combined with a register scheduling strategy specifically designed for the constraints of the Cortex-M4 architecture. We evaluate our implementation on the Nucleo-L4R5ZI evaluation board and compare it against existing state-of-the-art implementations. Our results demonstrate an 8% improvement in polynomial multiplication for BIKE, leading to up to 6% performance gains in key generation, encapsulation, and decapsulation. Moreover, we achieve an 80%–92% speedup in polynomial multiplication for HQC compared to the PQClean implementation, resulting in overall performance gains of up to 84% in key generation, encapsulation, and decapsulation.**

***Index Terms*—bit flipping key encapsulation (BIKE), Cortex-M4, Frobenius FFT, hamming quasi-cyclic (HQC), implementation, polynomial multiplication, post-quantum cryptography (PQC).**

I. Introduction

After the post-quantum cryptography (PQC) standardization project initiated by NIST in 2017, NIST announced four algorithms to be standardized in 2022 and prompted the continuation of round 4 on key encapsulation mechanism (KEM) to ensure diversity. The candidates for round 4 are Classic McEliece [1], bit flipping key encapsulation (BIKE) [2], hamming quasi-cyclic (HQC) [3], and SIKE [4], but SIKE has been excluded from the list due to vulnerabilities. The remaining three algorithms are all code-based cryptosystems. Ultimately, HQC was selected for standardization.

Code-based cryptography started with the introduction of McEliece in 1978, which used binary Goppa codes [5]. McEliece is based on the hardness of decoding general linear codes, which is known to be NP-hard. Over the years, various variants using different linear codes have been proposed, but only McEliece with Goppa codes has remained resistant to cryptanalysis. The round 4 candidate, Classic McEliece, is a code-based KEM that uses a binary Goppa code in the Niederreiter variant of the McEliece cryptosystem combined with standard techniques to achieve CCA security. Classic McEliece is also known to have efficient performance as the main operations of Classic McEliece can be considered matrix-vector multiplications over $\mathbb{F}_2$. On the other hand, Classic McEliece, or code-based cryptography in general, suffers from an extremely large key size. In this regard, many researchers have conducted studies to solve this problem, and one of them is the use of quasi-cyclic moderate density parity check codes (QC-MDPC codes).

BIKE, proposed in [2], follows the structure of the McEliece scheme instantiated with QC-MDPC codes, using the equivalent Niederreiter scheme. Its security is based on the hardness of the syndrome decoding problem in the Hamming metric for quasi-cyclic codes and the indistinguishability of the public key from a random element. Compared to Classical McEliece, BIKE offers a good balance between security, key size, and computational efficiency. HQC , proposed in [3], is based on the difficulty of decoding random quasi-cyclic codes. Its security relies on the hardness of decoding random quasi-cyclic codes in the Hamming metric. The scheme uses a concatenated coding structure, integrating an internal duplicated Reed-Muller code with an outer Reed-Solomon code.

To summarize, BIKE and HQC are proposed algorithms designed to address the drawback of Classic McEliece's large key size. Classic McEliece is ideal when storage is not a concern and long-term security is a priority. Although slightly larger than those of structured lattice KEMs, the quasi-cyclic structure of BIKE allows for public key and ciphertext sizes that are approximately 70% and 30%, respectively, of those of HQC. While BIKE is considered a promising candidate, its decryption failure rate remains insufficiently analyzed. In

Received 18 April 2025; revised 29 May 2025; accepted 16 June 2025. Date of publication 25 June 2025; date of current version 25 August 2025. This work was supported by the Samsung Electronics Company, Ltd., under Grant IO201209-07857-01. *(Corresponding author: Suhri Kim.)*

Myeonghoon Lee, Jihoon Jang, and Seokhie Hong are with the School of CyberSecurity, Korea University, Seoul 02841, Republic of Korea.

Suhri Kim is with the School of Mathematics, Statistics and Data Sciences, Sungshin Women's University, Seoul 02844, Republic of Korea (e-mail: suhrikim@sungshin.ac.kr).

Digital Object Identifier 10.1109/JIOT.2025.3583021

2327-4662 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: DGIST. Downloaded on March 01,2026 at 08:43:07 UTC from IEEE Xplore. Restrictions apply.

contrast, the security analysis of HQC is regarded as more mature and stable. Consequently, HQC was ultimately selected for standardization [6].

Meanwhile, since PQC algorithms are intended to replace currently deployed public-key cryptosystems such as RSA and ECC, they are expected to achieve efficiency that meets or exceeds that of existing schemes. As a result, extensive research has focused on optimization. In particular, because public-key schemes are designed based on mathematically hard problems, how the base algorithm of the underlying hardness assumption is implemented significantly affects overall performance. For code-based cryptography, most schemes are defined over the binary field $\mathbb{F}_2$. This stems from the historical development of coding theory, which primarily focused on correcting bit-level errors in digital communication systems. Since messages in such systems are transmitted as sequences of bits, linear codes—such as BCH and Reed–Solomon codes—have traditionally been constructed over $\mathbb{F}_2$. As a result, modern code-based cryptosystems, including McEliece, BIKE, and HQC, are naturally built upon binary codes. Consequently, polynomial multiplication over $\mathbb{F}_2[x]$ becomes a fundamental primitive in their implementations. Therefore, optimizing polynomial multiplication over $\mathbb{F}_2$ is essential for practical performance.

A. Motivation

According to the NIST PQC standardization proposal [7] published in 2016, algorithm performance is the second most important evaluation criterion. In the NIST-IR-8413 document [8] published in 2022, two benchmarking platforms were proposed: 1) x86-64 processors with AVX2 extensions and 2) the ARM Cortex-M4. In CHES 2021, Chen and Chou [9] and Chen et al. [10]implemented Classic McEliece and BIKE on the ARM Cortex-M4. However, HQC has not yet been implemented on the ARM Cortex-M4.

HQC uses polynomial rings of degrees over 10 000. Therefore, the time required for polynomial multiplication takes up a significant portion of the overall algorithm. There are three main polynomial multiplication algorithms over $\mathbb{F}_2$, such as Karatsuba, Toom-Cook, and Additive FFT. The first two algorithms can be used regardless of the degree of the polynomial. However, they have a multiplication complexity of $O(n^{\log_k(2k-1)})$, which is somewhat higher than that of Additive FFT. On the other hand, Additive FFT has a lower multiplication complexity of $O(n \log n)$, but it is known only to be applicable when the degree of the polynomial is a power of 2. According to the results from [10], the BIKE algorithm performs multiplication more efficiently using the Toom-Cook algorithm when AVX2 extensions are available. However, on embedded systems that use processors such as the Cortex-M4, utilizing Additive FFT for multiplication is more efficient for implementing BIKE.

B. Contributions

This work aims to efficiently implement the Frobenius Additive FFT on the Cortex-M4 platform and evaluate its performance by applying it to HQC and BIKE. The following list details the main contributions of this work.

1) To reduce the XOR count during the Frobenius Additive FFT, this article introduces a novel optimization technique that transforms field multiplication into a binary matrix multiplication. We use the fact that one of the multipliers in the field multiplication required for Frobenius Additive FFT is fixed. This observation allows us to recast the multiplication as a binary matrix-vector product, analogous to the linear layer in symmetric cryptographic primitives. Then, we employ the tool from [11] to find an optimized XOR sequence, achieving near-minimal gate complexity for this critical operation. However, due to the limited number of registers on the target board, an efficient method was needed to implement the XOR sequence we found. We propose a register scheduling strategy that optimizes resource utilization. As a result, we achieve fixed-input multiplication with approximately 30% fewer clock cycles compared to previous research.
2) We applied our proposed optimization technique to polynomial multiplication in the BIKE algorithm. As a result, compared to the state-of-the-art implementation, our approach achieves approximately 8% better performance in polynomial multiplication for both BIKE-1 and BIKE-3. Additionally, in the overall execution of the BIKE algorithm, we observed performance improvements of 6% in key generation, 3% in encapsulation, and 2% in decapsulation for both BIKE-1 and BIKE-3.
3) We present the first implementation of HQC on the Cortex-M4. While previous optimization efforts for HQC have focused on general-purpose (GP) processors or hardware accelerators [12], [13], no prior work has targeted embedded platforms. However, evaluating HQC on resource-constrained devices like the Cortex-M4 is essential for practical deployment. We benchmarked our implementation against a 32-bit ported version of the PQClean reference code [14], which was originally designed for 64-bit processors. Compared to the PQClean, our results show up to a 92% improvement in polynomial multiplication and overall speedups of 70%, 77%, and 84% for HQC-128, HQC-192, and HQC-256, respectively.

All the source code of the implementation is available at: https://github.com/myhoon/FAFFT_HQC.

C. Application

Sensor nodes are integral components in IoT systems, responsible for collecting environmental data and securely transmitting it to central servers or gateways. These nodes typically consist of a sensing module, a wireless communication interface, and a low-power microcontroller that handles signal processing and security functions. A representative example is the K32W0x series by NXP, which integrates a Cortex-M4 core along with Bluetooth Low Energy and IEEE 802.15.4 radios, making it well-suited for wireless sensor networks in smart home environments [15]. When the K32W0x series

Authorized licensed use limited to: DGIST. Downloaded on March 01,2026 at 08:43:07 UTC from IEEE Xplore. Restrictions apply.

is used for security-critical applications such as firmware signature verification or device authentication, currently used public-key algorithms, such as RSA or ECC, can no longer be used due to their vulnerability to quantum attacks. Therefore, implementing PQC algorithms on this platform is essential to ensure long-term security in the presence of quantum adversaries.

This work presents an optimized implementation of code-based cryptography targeting the Cortex-M4 architecture. By tailoring our implementation to the constraints of these platforms, we demonstrate that post-quantum cryptographic schemes can be securely and efficiently deployed even in resource-constrained sensor nodes. This highlights the practical applicability of our work for lightweight and secure PQC integration in real-world IoT environments.

D. Organization

The remainder of this article is organized as follows. In Section II, we explain how to perform Additive FFT on polynomials of arbitrary degree. In Section III, we describe the Frobenius Additive FFT, which performs FFT with fewer point evaluations than the Additive FFT. In Section IV, we present a new perspective for optimizing the Frobenius Additive FFT and propose an efficient implementation technique for the ARM Cortex-M4. In Section V, we present the results of our optimized implementation and compare its performance with the existing implementation. Finally, we conclude this article in Section VI.

II. Preliminaries

A. Additive FFT on Arbitrary Degree

In this section, we discuss how to perform polynomial multiplication over $\mathbb{F}_2[x]/(x^n - 1)$ using the Additive FFT, where n is an arbitrary integer.

For given polynomials $A(x), B(x) \in \mathbb{F}_2[x]/(x^n - 1)$, $C(x) = A(x)B(x) \in \mathbb{F}_2[x]/(x^n - 1)$ is calculated as follows.

1) Find the smallest l such that l is a power of 2 and $l > 2n-2$. The coefficients of the polynomial are considered as elements of $\mathbb{F}_{2^m}$, where $2^m \geq l$ and m is a power of 2. In simple terms, $A(x)$ and $B(x)$ are considered as polynomials over $\mathbb{F}_{2^m}[x]/(x^l - x)$.
2) Compute the values of $A(x)$ and $B(x)$ at l points in $\mathbb{F}_{2^m}$ using the Additive FFT.
3) Perform pointwise multiplications for the l values of $A(x)$ and $B(x)$.
4) Recover the polynomial $C'(x) \in \mathbb{F}_{2^m}[x]/(x^l - x)$ from l values using the inverse Additive FFT.
5) Reduce $C'(x) \in \mathbb{F}_{2^m}[x]/(x^l - x)$ to $C(x) \in \mathbb{F}_2[x]/(x^n - 1)$. At here, $C'(x)$ is the product of the $A(x)$ and $B(x)$, so the coefficients of $C'(x)$ are all in $\mathbb{F}_2$.

B. Tower of Fields

This section describes how to extend the binary field $\mathbb{F}_2$ to $\mathbb{F}_{2^m}$, where $m = 2^{l_m}$, with a focus on the specific case of $m = 32$ as used in our work.

Construct $\mathbb{F}_{2^2}$ using new basis x_1 such that $\mathbb{F}_2[x_1]/(x_1^2 + x_1 + 1)$. Similarly, construct $\mathbb{F}_{2^4}$ using x_2 such that $\mathbb{F}_{2^2}[x_2]/(x_2^2 + x_2 + x_1)$. Recursively, $\mathbb{F}_{2^{32}}$ is constructed as follows:

$$\begin{aligned}
\mathbb{F}_{2^2} &:= \mathbb{F}_2[x_1]/\left(x_1^2 + x_1 + 1\right)\\
\mathbb{F}_{2^4} &:= \mathbb{F}_{2^2}[x_2]/\left(x_2^2 + x_2 + x_1\right)\\
\mathbb{F}_{2^8} &:= \mathbb{F}_{2^4}[x_4]/\left(x_4^2 + x_4 + x_2x_1\right)\\
\mathbb{F}_{2^{16}} &:= \mathbb{F}_{2^8}[x_8]/\left(x_8^2 + x_8 + x_4x_2x_1\right)\\
\mathbb{F}_{2^{32}} &:= \mathbb{F}_{2^{16}}[x_{16}]/\left(x_{16}^2 + x_{16} + x_8x_4x_2x_1\right).
\end{aligned}$$

As denoted above, tower fields have nice properties when performing multiplication. To multiply $a, b \in \mathbb{F}_{2^{32}}$, a and b are splited into two parts $a = a_0 + a_1x_{16}$ and $b = b_0 + b_1x_{16}$. The Karatsuba algorithm is then applied by computing the three intermediate products a_0b_0, a_1b_1, and $(a_0 + a_1)(b_0 + b_1)$ in $\mathbb{F}_{2^{16}}$. Each of these products is further computed recursively using the same Karatsuba technique.

C. Cantor Basis

In 1989, Cantor introduced a special basis called Cantor basis [16] for computing the FFT over $\mathbb{F}_{2^m}$ where m is a power of 2. Cantor basis $\{v_0, v_1, \ldots, v_{m-1}\} \in \mathbb{F}_{2^m}$ is as follows:

$$v_0 = 1, \quad v_{i+1}^2 + v_{i+1} = v_i, \quad \text{for} \quad 0 \leq i < m-1.$$

Define $W_k \subseteq \mathbb{F}_{2^m}$ as the span of $\{v_0, v_1, \ldots, v_{k-1}\}$ for $1 \leq k < m$ and $W_0 = \{0\}$. For $k \geq 1$, we represent $(i+1)$th elements of W_k as $\omega_i = i_0v_0 + i_1v_1 + \cdots + i_{k-1}v_{k-1}$, where $i = i_0 + 2i_1 + 2^2i_2 + \cdots + 2^{k-1}i_{k-1}$. We also define a series of polynomials $s_i(x)$

$$s_0(x) = x, \quad s_{i+1}(x) = (s_i(x))^2 + s_i(x), \quad \text{for} \quad i \geq 0.$$

Then, v_i, W_i, and $s_i(x)$, where $0 \leq i < m$ have the following properties.

1) $\deg(s_i(x)) = |W_i| = 2^i$.
2) $s_i(x+y) = s_i(x) + s_i(y)$, and $s_k(x) = x^{2^k} + x$ if and only if k is a power of 2.
3) $s_k(x) = \prod_{a \in W_k}(x - a)$. we called $s_k(x)$ as subspace polynomials.
4) $s_k(v_i) = \begin{cases} v_{i-k}, & \text{for } i > k \\ 1, & \text{for } i = k \\ 0, & \text{for } i < k \end{cases}$.

D. Additive FFT Using Cantor Basis

In this section, we show how to evaluate l points of $f(x) \in \mathbb{F}_{2^m}[x]/(x^l - x)$ using Cantor basis. First, we represent the polynomial in terms of a basis related to the $s_i(x)$'s, which are called a novel polynomial basis. Then, we perform the butterfly operation to evaluate l points.

We first describe the Additive FFT algorithm (denoted as `AddFFT`) in Algorithm 1, where `BasisCvt` (Basis conversion) and `Btfy` (Butterfly operations) are explained in the following sections.

Authorized licensed use limited to: DGIST. Downloaded on March 01,2026 at 08:43:07 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Additive FFT Algorithm

1: `AddFFT`$(f(x), \alpha \in \mathbb{F}_{2^m})$:
Input: $f(x) = f_0 + f_1 x + \cdots + f_{n-1}x^{n-1}, \alpha \in \mathbb{F}_{2^m}$.
Output: The values for evaluating f at $\alpha + W_{l_n}$.
2: Compute $g(X) \leftarrow$ `BasisCvt`$(f(x))$.
3: **return** `Btfy`$(g(X), \alpha)$

1) Basis Conversion to Novel Polynomial Basis: Lin et al. [17] proposed a new polynomial basis called the novel polynomial basis. They define the polynomials $X_k(x)$ as follows:

$$X_k(x) = \prod_i (s_i(x))^{b_i}.$$

where k is expressed in binary as

$$k = \sum b_i 2^i, \quad \text{with } b_i \in \{0, 1\}.$$

Since $b_i = 1$ only when the ith bit of k is 1, the degree of each contributing term is 2^i. Consequently, the degree of $X_k(x)$ is given by

$$\deg(X_k(x)) = \sum b_i \cdot 2^i = k.$$

Thus, the polynomial $X_k(x)$ has degree exactly k, aligning with its binary representation. Using this basis, a polynomial $f(x)$ can be represented as

$$f(x) = g(X) = g_0 + g_1 X_1(x) + \cdots + g_{n-1}X_{n-1}(x),$$

where $g_i \in \mathbb{F}_{2^m}$. Lin et al. [18] proposed an efficient basis conversion algorithm that takes advantage of the structural properties of the polynomial $s_k(x)$, such that $s_k(x) = x^{2^k} + x$ where k is a power of two. The algorithm proceeds as follows.

1) Find the largest $i = 2^k$ such that $\deg(s_i) < n$.
2) Divide the polynomial $f(x)$ by $s_i(x)$, which gives $f(x) = f_0(x) + s_i(x) f_1(x)$.
3) Recursively divide $f_0(x)$ and $f_1(x)$ until they become constants.

The detailed algorithm is described in Algorithm 2, denoted as `BasisCvt`. Since the coefficients of $f(x)$ are all in $\mathbb{F}_2$, the `BasisCvt` process performs with only `XOR` operations. The computational complexity of the conversion process is $O(n \log_2 n \log_2(\log_2 n))$.

2) Butterfly Operations: Through the `BasisCvt` process, the polynomial $g(X)$, represented in the novel polynomial basis, can efficiently evaluate $l = 2^{l_n}$ points over the set $\alpha + W_{l_n} = \{\alpha + \omega : \alpha \in \mathbb{F}_{2^m}, \omega \in W_{l_n}\}$ using the butterfly operations, where $l = 2^{l_n}$. The detailed algorithm is as follows in Algorithm 3, denoted as `Btfy`.

The `Btfy` algorithm recursively divides the polynomial $g(X)$ into two parts, $p_0(X)$ and $p_1(X)$, and then computes $h_0(X)$ and $h_1(X)$ (lines 7 and 8 in Algorithm 3). Since $s_i(v_i) = 1$ is in the Cantor basis, each `Btfy` iteration requires only 2 field additions and 1 field multiplication.

III. Frobenius Additive FFT

In CHES2021, Chen et al. [10] applied the Frobenius Additive FFT algorithm (denoted as `FAFFT`) using the Frobenius map to optimize BIKE on the ARM Cortex-M4. The `FAFFT` evaluates fewer points than `AddFFT`. The `FAFFT` algorithm consists of three steps, such as `BasisCvt`, `Encode`, and `Btfy`. In this section, we first explain the Frobenius map and Frobenius partition, followed by the `Encode` process of `FAFFT`. Finally, we provide a detailed explanation of `FAFFT` at the end of this section.

Algorithm 2 Basis Conversion: Monomial Basis to Novel Polynomial Basis

1: `BasisCvt`$(f(x))$:
Input: $f(x) = f_0 + f_1 x + \cdots + f_{n-1}x^{n-1}$, which is monomial basis.
Output: $g(X) = g_0 + g_1 X_1(x) + \cdots + g_{n-1}X_{n-1}(x)$, which is novel polynomial basis representation.
2: **if** $\deg(f(x)) \leq 1$ **then**
3: **return** $g(X) \leftarrow f_0 + f_1 X_1$
4: **end if**
5: Let $i \leftarrow \mathrm{Max}(2^k)$ where $k \in \mathbb{N}$ s.t. $\deg(s_i(x)) \leq \deg(f(x))$.
6: Compute $h(x, y) = \hat{h}(y) = h_0(x) + h_1(x)y + \cdots + h_{m-1}(x)y^{m-1}$ s.t. $f(x) = h(x, s_i(x))$. At here, degree of $h_0(x), \cdots, h_{m-1}(x) < 2^i$.
7: Compute $h'(Y) = q_0(x) + \cdots + q_{m-1}(x)X_{(m-1)\cdot 2^k} \leftarrow$ `BasisCvt`$(\hat{h}(y))$.
8: Compute $g_i(X) \leftarrow$ `BasisCvt`$(q_i(x))$ for all coefficients $q_i(x)$ of $h'(Y)$.
9: **return** $g(X) = g_0(X) + g_1(X)X_{2^k} + \cdots + g_{n-1}(X)X_{(m-1)\cdot 2^k}$

Algorithm 3 Butterfly Operations

1: `Btfy`$(g(X), \alpha \in \mathbb{F}_{2^m})$:
Input: $g(X) = f(x)$, which is novel polynomial basis representation, $\alpha \in \mathbb{F}_{2^m}$.
Output: A list: $[f(\omega_0+\alpha), f(\omega_1+\alpha), \ldots, f(\omega_{2^l-1}+\alpha)]$, which is the evaluation of f at $\alpha + W_{l_n}$.
2: **if** $\deg(g(X)) = 0$ **then**
3: **return** $[g_0]$
4: **end if**
5: Let $i \leftarrow \lfloor \log_2 l \rfloor - 1$ such that $\deg(s_i(x)) = \deg(X_{2^i}(x)) = 2^i < l$.
6: Let $g(X) = p_0(X) + X_{2^i}(x) \cdot p_1(X) = p_0(X) + s_i(x) \cdot p_1(X)$.
7: Compute $h_0(X) \leftarrow p_0(X) + s_i(\alpha) \cdot p_1(X)$.
8: Compute $h_1(X) \leftarrow h_0(X) + s_i(v_i) \cdot p_1(X)$.
9: **return** $[$`Btfy`$(h_0(X), \alpha)$, `Btfy`$(h_1(X), v_i + \alpha)]$

A. Frobenius Map and Frobenius Partition

Let $\phi_2 : x \mapsto x^2$ be the Frobenius map over $\mathbb{F}_2$. If the value of the polynomial $f(x) \in \mathbb{F}_{2^m}[x]/(x^l - x)$ is known at $a \in \mathbb{F}_{2^m}$, i.e., $f(a)$, then the value of $f(a^2)$ can be derived as $f(\phi_2(a)) = \phi_2(f(a))$. Using this property, we define the order of the Frobenius map and the Frobenius partition Σ, which were first introduced in [19].

Definition 1 (Order of the Frobenius Map): Given a set Σ, the order of the Frobenius map ϕ_2 is defined as the smallest

Authorized licensed use limited to: DGIST. Downloaded on March 01,2026 at 08:43:07 UTC from IEEE Xplore. Restrictions apply.

integer $j \in \mathbb{N}$ such that

$$\Sigma = \phi_2^j(\Sigma).$$

That is, $\mathrm{Ord}_{\phi_2}(\Sigma) = j$ is the minimal j for which applying ϕ_2 repeatedly to Σ returns to the original set.

Definition 2 (Frobenius Partition): Given $\mathrm{Ord}_{\phi_2}(\Sigma) = j$, we call Σ a partition of Ω under the Frobenius map if

$$\Omega = \Sigma \cup \phi_2(\Sigma) \cup \cdots \cup \phi_2^{(j-1)}(\Sigma)$$

and all Σ, $\phi_2(\Sigma)$, ..., and $\phi_2^{(j-1)}(\Sigma)$ are disjoint sets.

For the polynomial $f(x) \in \mathbb{F}_{2^m}[x]/(x^l - x)$, the values at Ω can be derived from the values at Frobenius partition Σ. Li et al. [19] showed some properties between the Frobenius partition and the Cantor basis for FAFFT.

1) $\mathrm{Ord}_{\phi_2}(v_i) = 2 \times 2^{\lfloor \log_2 i \rfloor}$ for $i > 0$.
2) $\phi_2(W_i) = W_i$.
3) $\Sigma = v_{k+m/2} + W_k$, where $k = \log_2 l - \log_2 m < (m/2)$ and $l \geq 0$, is a Frobenius partition.
4) $\mathrm{Ord}_{\phi_2}(\Sigma) = m$, $|\Sigma| = 2^k = l/m$.
5) For $\Omega = \Sigma \cup \phi_2(\Sigma) \cup \cdots \cup \phi_2^{(m-1)}(\Sigma)$, $|\Omega| = l$.

Using these properties, the polynomial $f(x)$ can be evaluated at 2^k points of the Frobenius partition $\Sigma = v_{k+m/2} + W_k$, allowing us to derive l points in Ω. For evaluation at Σ, where $|\Sigma| = 2^k$, we truncate the upper half of the polynomial $f(x)$ during the first $l_m = \log_2 m$ iterations of the Btfy process. This process is referred to as Encode.

B. Encode

To evaluate at $\Sigma = v_{k+m/2} + W_k$, we perform Encode process before Btfy process. The difference between the Encode process and the Btfy process is that the Encode process does not compute line 8 of Algorithm 3. For the first l_m layers, we only compute line 7 of Algorithm 3 for $\alpha = v_{k+m/2}$ and $s_i(\alpha)$, where $i = l_m - 1, \ldots, k = l_n - l_m$. Therefore, the multipliers of $s_i(\alpha)$ are given by

$$\left(s_{l_n-1}\left(v_{k+m/2}\right), \ldots, s_k\left(v_{k+m/2}\right)\right) = \left(v_{m/2-l_m+1}, \ldots, v_{m/2}\right).$$

For a given m, $(v_{m/2-l_m+1}, \ldots, v_{m/2})$ are always the same. Therefore, for the given Cantor basis $\{v_i\} \in \mathbb{F}_{2^m}$, we can determine the result polynomial $A(X) = a_0 + a_1X_1 + \cdots + a_{2^k-1}X_{2^k-1}$ of the Encode process for the polynomial $g(X) = g_0 + g_1X_1 + \cdots + g_{l-1}X_{l-1}$. First, we define $\{r_0, \ldots, r_{m-1}\}$ as follows:

$$r_j = \prod_{i=0}^{l_m-1} \left(v_{m/2-i}\right)^{j_i}, \text{ for } j = j_0 + j_1 \cdot 2 + \cdots + j_{l_m-1} \cdot 2^{l_m-1}.$$

Then, the coefficients of the result polynomial $A(X)$ are given by

$$a_i = \sum_{j=0}^{m-1} g_{j \cdot 2^k + i} \cdot r_j.$$

The detailed algorithm is as follows in Algorithm 4.

Algorithm 4 Encode Process

Input: $g(X) = g_0 + g_1X_1 + \cdots + g_{l-1}X_{l-1}$.
Output: $A(X) = a_0 + a_1X_1 + \cdots + a_{2^k-1}X_{2^k-1}$, where $2^k = l/m$.

1: **for** $i \in \{0, \ldots, 2^k - 1\}$ **do**
2: Collect $\left(g_{i+0\cdot 2^k}, g_{i+1\cdot 2^k}, \ldots, g_{i+(m-1)\cdot 2^k}\right)$.
3: Compute $a_i \leftarrow \sum_{j=0}^{m-1} g_{j\cdot 2^k+i} \cdot r_j$.
4: **end for**
5: **return** $(a_0, a_1, \ldots, a_{2^k-1})$

Algorithm 5 Polynomial Multiplication Using FAFFT

Input: $A(x) = a_0 + \cdots + a_{n-1}x^{n-1}$,
$B(x) = b_0 + \cdots + b_{n-1}x^{n-1}$.
Output: $C(x) = A(x) \cdot B(x) = c_0 + \cdots + c_{2n-2}x^{2n-2}$.
// $A(x), B(x) \in \mathbb{F}_{2^m}[x]/(x^l - x)$, where $l > 2n - 2$.

1: Compute $A(X) \leftarrow$ BasisCvt$(A(x))$.
2: Compute $f(X) \leftarrow$ Encode$(A(X))$.
3: Compute $\left(\hat{a}_0, \cdots, \hat{a}_{2^k-1}\right) \leftarrow$ Btfy$(f(X), v_{k+m/2})$.
4: Compute $B(X) \leftarrow$ BasisCvt$(B(x))$.
5: Compute $g(X) \leftarrow$ Encode$(B(X))$.
6: Compute $\left(\hat{b}_0, \cdots, \hat{b}_{2^k-1}\right) \leftarrow$ Btfy$(g(X), v_{k+m/2})$.
7: Compute $\hat{C} \leftarrow \left(\hat{c}_0 = \hat{a}_0 \cdot \hat{b}_0, \cdots, \hat{c}_{2^k-1} = \hat{a}_{2^k-1} \cdot \hat{b}_{2^k-1}\right)$.
8: Compute $h(X) \leftarrow$ iBtfy$(\hat{C}, v_{k+m/2})$.
9: Compute $C(X) \leftarrow$ Decode$(h(X))$.
10: Compute $C(x) \leftarrow$ iBasisCvt$(C(X))$.
11: **return** $C(x)$

C. FAFFT and Polynomial Multiplication

Using Frobenius partition $\Sigma = v_{k+m/2} + W_k$ and Encode process, we can perform FAFFT to polynomial multiplication.

For two given polynomials $A(x) = a_0 + a_1x + \cdots + a_{n-1}x^{n-1}$ and $B(x) = b_0 + b_1x + \cdots + b_{n-1}x^{n-1}$ over $\mathbb{F}_2[x]/(x^n - 1)$, BasisCvt is applied to each polynomial, treating $A(X)$ and $B(X)$ as polynomials over $\mathbb{F}_{2^m}[x]/(x^l - x)$, where $l > 2n - 2$. Then, Encode is performed for the first l_m iterations of the Btfy process. The remaining iterations of Btfy are initiated at $\alpha = v_{k+m/2}$, with α updated recursively at each level. The resulting evaluation points $(\hat{a}_0, \ldots, \hat{a}_{2^k-1})$ and $(\hat{b}_0, \ldots, \hat{b}_{2^k-1})$ from the Btfy process correspond to the outputs of the FAFFT applied to $A(x)$ and $B(x)$.

For polynomial multiplication, pointwise multiplication is performed on $\hat{a}_i$ and $\hat{b}_i$. The result of the pointwise multiplication, $\hat{C} = (\hat{c}_0 = \hat{a}_0 \cdot \hat{b}_0, \ldots, \hat{c}_{2^k-1} = \hat{a}_{2^k-1} \cdot \hat{b}_{2^k-1})$, corresponds to the FAFFT output of the polynomial $C(x) = A(x) \cdot B(x)$. The polynomial $C(x)$ is recovered by applying iBtfy, Decode, and iBasisCvt, which are the inverse operations of Btfy, Encode, and BasisCvt, respectively. The complete polynomial multiplication procedure using FAFFT is summarized in Algorithm 5.

IV. Proposed Optimization Method of Frobenius Additive FFT

In this section, we propose an optimization strategy for implementing FAFFT on the Cortex-M4. The focus was placed

Authorized licensed use limited to: DGIST. Downloaded on March 01,2026 at 08:43:07 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PARAMETERS FOR BIKE AND HQC

Algorithm	n	l	m	k	Σ
BIKE-1	12323	32768	32	10	$v_{26}+W_{10}$
BIKE-3	24659	65536	32	11	$v_{27}+W_{11}$
BIKE-5	40973	131072	32	12	$v_{28}+W_{12}$
HQC-128	17669	65536	32	11	$v_{27}+W_{11}$
HQC-192	35851	131072	32	12	$v_{28}+W_{12}$
HQC-256	57637	131072	32	12	$v_{28}+W_{12}$

on the `Btfy` the most computation-intensive step in `FAFFT`. After the `BasisCvt`, `Encode` is processed in order to convert the coefficients in $\mathbb{F}_2$ to $\mathbb{F}_{2^m}$, which results in the output being represented in a bit-sliced form. This bit-sliced output is then fed into `Btfy`. Due to the use of bit-sliced representation and the fact that the Cantor basis is fixed, we observed that the operation in `Btfy` can be reformulated as a binary matrix multiplication. Furthermore, we noticed that the corresponding binary matrix multiplication closely resembles the linear layer in symmetric-key ciphers. To the best of our knowledge, we are the first to apply linear layer optimization techniques from symmetric-key cryptography to binary matrix multiplication in this context, enabling efficient computation through structural similarity. Following this, we propose a register scheduling technique to efficiently implement the optimized binary matrix multiplication on the target platform, Cortex-M4.

To describe our method, we first demonstrate how polynomials are transformed into the bit-slice representation in the `Encode` step. Then, we present a detailed explanation of the corresponding optimization for `Btfy`.

Since our primary focus is polynomial multiplication of BIKE and HQC, we consider only the Cantor basis $\{v_0, v_1, \ldots, v_{31}\}$, where $m = 32$. Moreover, to better compare our optimization strategy, we use the same Cantor basis as in [10], and v_{31} is defined as follows:

$$\begin{aligned} v_{31} = {} & 1 + x_1x_2 + x_1x_2x_4 + x_8 + x_1x_2x_8 + x_1x_4x_8 + x_2x_4x_8 \\ & + x_{16} + x_1x_2x_{16} + x_4x_{16} + x_1x_4x_{16} + x_2x_4x_{16} \\ & + x_1x_2x_4x_{16} + x_1x_8x_{16} + x_4x_8x_{16} + x_1x_2x_4x_8x_{16}. \end{aligned}$$

The parameters used to apply `FAFFT` to the polynomial $f(x) \in \mathbb{F}_2[x]/(x^n - 1)$ in the BIKE and HQC algorithms are shown in Table I. BIKE-1, –3, and –5 are the parameter sets for security levels 1, 3, and 5, respectively. Similarly, HQC–128, –192, and –256 are the parameter sets for security levels 1, 3, and 5, respectively.

A. Bitslice Implementation

We first check the polynomial after the `BasisCvt` process. Let $g(X)$ be the polynomial obtained by applying the `BasisCvt` process to $f(x)$. Since the coefficients of the polynomial $f(x)$ are all in $\{0, 1\}$, the coefficients of $g(X)$ are also all in $\{0, 1\}$. Then, the `Encode` (Algorithm 4) is used to convert coefficients in $\mathbb{F}_2$ to $\mathbb{F}_{2^m}$. In our implementation, we use $m = 32$. For $m = 32$, the process of encoding the polynomial $g(X)$ is as follows for $\{r_0, \ldots, r_{31}\}$:

$$r_j = \prod_{i=0}^{4} (v_{16-i})^{j_i}, \quad \text{for} \quad j = j_0 + j_1 \cdot 2 + \cdots + j_4 \cdot 2^4.$$

Then, the coefficients of the result polynomial $A(X)$ are given by

$$a_i = \sum_{j=0}^{31} g_{j \cdot 2^k + i} \cdot r_j.$$

Since the values of the Cantor basis are known, we can precompute the values of $\{r_0, \ldots, r_{31}\}$. Additionally, r_j's are elements of $\mathbb{F}_{2^{32}}$, and $g_{j \cdot 2^k + i}$ is either 0 or 1. Therefore, the value of $g_{j \cdot 2^k + i} \cdot r_j$ is either 0 or r_j, and the value of a_i can be computed as the XOR of the corresponding r_j values. The value of $a_i \in \mathbb{F}_{2^{32}}$ can be represented by the following matrix multiplication:

$$\begin{bmatrix} a_{i,0} \\ a_{i,1} \\ \vdots \\ a_{i,30} \\ a_{i,31} \end{bmatrix} = \begin{bmatrix} r_0 & r_1 & \cdots & r_{30} & r_{31} \end{bmatrix} \begin{bmatrix} g_i \\ g_{2^k+i} \\ \vdots \\ g_{30 \cdot 2^k + i} \\ g_{31 \cdot 2^k + i} \end{bmatrix}. \quad (1)$$

Here, $a_{i,j}$ for $j = 0, \ldots, 31$ are jth value of a_i. The coefficients of the polynomial $g(X)$ are stored consecutively, and for $0 \le i < 2^k$, the same values $\{r_0, \ldots, r_{31}\}$ are multiplied in (1). Additionally, since our target device has a 32-bit architecture, 32 coefficients are stored in a single variable. Therefore, 32 values can be computed simultaneously. The following is an example of computing $(a_{31}, \ldots, a_0)$ for $k = 10$:

$$\begin{bmatrix} \overline{a_{31,0} \cdots a_{0,0}} \\ \overline{a_{31,1} \cdots a_{0,1}} \\ \vdots \\ \overline{a_{31,30} \cdots a_{0,30}} \\ \overline{a_{31,31} \cdots a_{0,31}} \end{bmatrix} = \begin{bmatrix} r_0 & r_1 & \cdots & r_{31} \end{bmatrix} \cdot \begin{bmatrix} \overline{g_{31}, \cdots g_0} \\ \overline{g_{2^{10}+31}, \cdots g_{2^{10}}} \\ \vdots \\ \overline{g_{30 \cdot 2^{10}+31}, \cdots g_{30 \cdot 2^{10}}} \\ \overline{g_{31 \cdot 2^{10}+31}, \cdots g_{31 \cdot 2^{10}}} \end{bmatrix}.$$

We collect the values of $g_{j \cdot 2^{10}+i}$ for $j = 0, \ldots, 31$ and perform pairwise XOR based on the values of $r_0, \ldots, r_{31}$. The resulting values, a_0 through a_{31}, are stored column-wise, which is in bitsliced form. Instead of rearranging the results, we perform further processing in a bitsliced manner.

B. Matrix Multiplication With Fixed Input

The field multiplication in `Btfy` is one of the most time-consuming operations in `FAFFT`. In the `Btfy` process, field multiplication is performed in line 7 of Algorithm 3, i.e., $s_i(\alpha) \cdot p_1(X)$.

Let $A(X)$ be the polynomial after the `Encode` process. In the `Btfy` process, the goal is to evaluate $A(X)$ at $v_{k+16} + W_k$, which is `Btfy`$(A(X), v_{k+16})$. At the first iteration, since $i = k - 1$, the value of $s_i(\alpha)$ is v_{17}. In the next iteration, `Btfy`$(h_0(X), v_{k+16})$ and `Btfy`$(h_1(X), v_{k-1} + v_{k+16})$ are computed. Then, $i = k - 2$, and the values of $s_i(\alpha)$ are v_{18} for $h_0(X)$ and $v_1 + v_{18}$ for $h_1(X)$, since $s_i(x + y) = s_i(x) + s_i(y)$. This implies that for every iteration of the `Btfy` process, the value of $s_i(\alpha)$ can be represented as the sum of v_i's, and

Authorized licensed use limited to: DGIST. Downloaded on March 01,2026 at 08:43:07 UTC from IEEE Xplore. Restrictions apply.

$s_i(v_{k+16}) = v_{k+16-i}$ is a common multiplier of both $h_0(X)$ and $h_1(X)$. Then, the sum of v_i's is of the form $\omega + v$, where $\omega \in W_k$ and $v \in \{v_{17}, v_{18}, \ldots, v_{k+16}\}$ is a common multiplier. Using the property that v is a common multiplier, we compute the values of $\omega \cdot p_1(X)$ and $v \cdot p_1(X)$ separately, then add the two values to obtain $(\omega + v)\ p_1(X)$.

For computing $\omega \cdot p_1(X)$, we use the same technique as in [10]. The multiplier $\omega \in W_k$ is always in a proper subfield of $\mathbb{F}_{2^{32}}$. For $\omega \in \mathbb{F}_{2^{m'}}$, instead of computing $\omega \cdot p_1(X)$ in $\mathbb{F}_{2^{32}}$, we perform the computation $32/m'$ times in the subfield $\mathbb{F}_{2^{m'}}$.

For computing $v \cdot p_1(X)$, we propose a new perspective, which is matrix multiplication with a fixed input. For the ith iteration of the `Btfy` process, the multiplier v is actually v_{16+i} for $i = 1, \ldots, k$. Suppose we perform $b = a \cdot v_{17}$, where $a, b \in \mathbb{F}_{2^{32}}$. Then, for $a_i \in \mathbb{F}_2$, a can be represented as

$$a = a_0 + a_1x_1 + a_2x_2 + a_3x_1x_2 + \cdots + a_{31}x_1x_2x_4x_8x_{16}.$$

Let

$$x_j = \prod_{i=0}^{4} (x_i)^{j_i}, \text{ for } j = j_0 + j_1 \cdot 2 + \cdots + j_4 \cdot 2^4.$$

Then, $b = a \cdot v_{17}$ is given by

$$b = a \cdot v_{17} = a_0v_{17} + a_1(v_{17}x_1) + a_2(v_{17}x_2) + \cdots + a_{31}(v_{17}x_{31}).$$

Since v_{17} is known, we can precompute $v_{17}x_1$, $v_{17}x_2$, $\cdots$, $v_{17}x_{31}$. Then, we can compute $b = a \cdot v_{17}$ using only XOR operations, similar to the `Encode` process. Now, we define the 32×32 binary matrix $V_{17} = [v_{17} \quad v_{17}x_1 \quad \cdots \quad v_{17}x_{30} \quad v_{17}x_{31}]$, where the ith column represents the value of $v_{17} \cdot x_{i-1}$, and the jth row corresponds to the jth value of $v_{17} \cdot x_{i-1}$. Then, we can represent $b = a \cdot v_{17}$ as follows:

$$\begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_{30} \\ b_{31} \end{bmatrix} = V_{17} \cdot \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{30} \\ a_{31} \end{bmatrix} = \begin{bmatrix} & & \\ v_{17} & \cdots & v_{17}x_{31} \\ & & \end{bmatrix} \cdot \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{30} \\ a_{31} \end{bmatrix}. \tag{2}$$

The components of the matrix V_{17} are elements of $\mathbb{F}_2$. Therefore, computing $b = a \cdot v_{17}$ is equivalent to performing the 32×32 binary matrix multiplication in (2). Similarly, we can define a 32×32 binary matrix V to compute $v \cdot p_1(X)$, allowing us to express the computation as a binary matrix multiplication.

1) Binary Matrix Multiplication: Our aim is to optimize the 32×32 binary matrix multiplication. Performing 32×32 binary matrix multiplication can be done using only XOR operations. Therefore, to optimize binary matrix multiplication, it is important to minimize the number of XOR operations. All binary matrices we use are invertible. Optimizing the multiplication of such invertible binary matrices is equivalent to performing linear layer operations in symmetric-key primitives. In symmetric-key primitives, extensive research has been conducted on optimizing linear layers to reduce the number of XOR operations, as seen in [20], [21], and [22]. In this context, we introduce two metrics, general XOR (g-XOR) and sequential XOR (s-XOR) related to the number of XOR operations.

1) *g-XOR:* For a given $m \times n$ binary matrix, the minimal number of XOR operations that completely compute the m outputs.
2) *s-XOR:* For a given $n \times n$ invertible binary matrix, the minimal number of XOR operations to update the inputs to the outputs.

We compute the output values based on the input values. Therefore, instead of focusing on g-XOR, we concentrate on the number of s-XOR counts, which is $in - place$ operations.

Meanwhile, the problem of finding the g-XOR and s-XOR counts required for performing the multiplication of large matrices, such as a 32×32 matrix, is equivalent to the shortest linear program (SLP) problem, which is NP-hard [23]. As a result, only heuristic algorithms exist, and we use the tool proposed in [11] to determine the minimal s-XOR count for the computation. Xiang et al. [11] proposed a technique to optimize the s-XOR count through matrix decomposition. By recognizing that each s-XOR corresponds to a type-3 elementary matrix in the decomposition of an invertible matrix over $\mathbb{F}_2$, they reduce the optimization problem to finding matrix decompositions with minimal use of such matrices. For details of the method, please refer to [11].

As the method proposed in [11] demonstrates superior or comparable performance to existing heuristics such as those by Paar [21] and Boyar and Peralta [24], we adopted the approach in [11] to optimize the multiplication of invertible binary matrices under the s-XOR metric. When applying this method, the resulting outputs appear in a permuted order. However, this output permutation can be handled without incurring additional cost by appropriately storing the values.

For optimization, the s-XOR count for $v \cdot p_1(X)$, where $v = v_{17}, \ldots, v_{28}$, is required. Table II presents the XOR counts for multiplication by v_i obtained using the tools from [11], and compares them with the results reported in [10]. As shown in Table II, the proposed method performs binary matrix multiplication using 49 to 76 fewer XOR operations compared to previous approaches.

TABLE II
XOR COUNT FOR MULTIPLICATION BY v_i

Multiplier	[10]	Ours	Difference
v_{17}	210	161	49
v_{18}	211	153	58
v_{19}	231	171	60
v_{20}	226	173	53
v_{21}	231	172	59
v_{22}	222	165	57
v_{23}	239	163	76
v_{24}	249	178	71
v_{25}	251	189	62
v_{26}	248	190	58
v_{27}	243	179	64
v_{28}	-	187	-

2) Applying on the Cortex-M4 and Comparison With Existing Implementation: Table II presents the s-XOR counts required for multiplication by v_i. However, the actual number

Authorized licensed use limited to: DGIST. Downloaded on March 01,2026 at 08:43:07 UTC from IEEE Xplore. Restrictions apply.

TABLE III
MULTIPLICATION COSTS IN TERMS OF CLOCK CYCLE

Multiplier	[10]	Ours	Difference
v_{17}	760	535	225
v_{18}	742	515	227
v_{19}	800	565	235
v_{20}	798	561	237
v_{21}	795	564	231
v_{22}	795	551	244
v_{23}	836	547	289
v_{24}	854	598	256
v_{25}	881	605	276
v_{26}	861	612	249
v_{27}	847	589	258
v_{28}	-	609	-

TABLE IV
PERFORMANCE COMPARISON OF POLYNOMIAL MULTIPLICATION FOR BIKE-1 AND BIKE-3

	[10]	**our code**
BIKE-1	1,598,737	1,475,953
BIKE-3	3,509,004	3,238,668

TABLE V
PERFORMANCE COMPARISON OF BIKE-1 AND BIKE-3 IMPLEMENTATIONS

	key gen.	**encap.**	**decap.**	**imple.**
BIKE-1	26,921,361	3,397,581	58,151,362	[10]
	24,915,790	3,274,791	56,840,529	**our code**
BIKE-3	65,320,819	8,542,735	159,610,301	[10]
	60,454,738	8,272,396	156,724,784	**our code**

of operations on the target platform, Cortex-M4, may differ from the s-XOR count due to its limited number of registers.

The Cortex-M4 has 16 GP registers, but only 14 of them are available for actual implementation. However, the s-XOR count we obtained represents the minimum number of XOR operations required to compute the output from 32 input values, meaning that the XOR operations cannot be performed all at once. On the other hand, the Cortex-M4 has 32 floating-point (FP) registers, allowing us to store all 32 input values. Therefore, we store all input value in the 32 FP registers and move the data required for XOR operations to the 14 available GP registers for computation.

On the Cortex-M4, an XOR operation requires only one clock cycle. The `vmov` instruction, which is needed to transfer data for the XOR operation, also requires one cycle. Therefore, to efficiently perform XOR operations, we minimized data exchanges between FP registers and GP registers. To achieve this, we implemented the XOR sequence obtained from [11] in the following order.

1) Load the 32 input values into 32 FP registers.
2) If a GP register is available, move the required value for the XOR operation from the FP register to the GP register.
3) If all GP registers are full, move the value with the farthest subsequent use from the GP register back to the FP register and load the required value into the GP register.
4) Once the entire sequence of XOR operations is completed, store the values in the GP and FP registers appropriately.

The XOR sequence we found is optimal in terms of XOR count, resulting in the 32 output values being permuted. However, since the permutation is known, the computation can be performed without additional cost as long as the storage order in step 4 is carefully maintained. The result of implementing the XOR sequence according to the above process is shown in Table III. As shown in Table III, our method achieves 225 to 289 fewer cycles compared to the results in [10].

V. IMPLEMENTATION RESULTS

In this section, we present the implementation results to verify the efficiency of the optimization methods proposed in Section IV. We use the NUCLEO-L4R5ZI evaluation board as our benchmarking platform, featuring an STM32L4R5ZI microcontroller. This board is based on an ARM Cortex-M4 core and includes 2MB of flash memory and 640KB of SRAM. While the system clock can run at a maximum of 120 MHz, we use 20 MHz, as specified by the `pqm4` framework [25] for zero wait states. The code is compiled using `arm-none-eabi-gcc 10.3.0`.

We apply the proposed optimization methods to BIKE and HQC, and compare the performance with [10] and [14], respectively. For each algorithm, we first present the implementation results to evaluate the improvements in polynomial multiplication. Then, we analyze how these improvements translate into overall performance gains in key generation, encapsulation, and decapsulation.

A. BIKE

Table IV shows the number of cycles for polynomial multiplication used in BIKE-1 and BIKE-3. Our new technique achieves approximately 8% better performance compared to previous results when performing polynomial multiplication in BIKE-1 and BIKE-3.

Table V shows the number of cycles required to perform key generation, encapsulation, and decapsulation in BIKE-1 and BIKE-3. We replaced only the polynomial multiplication implementation in Chen et al.'s BIKE implementation with our new approach, and we observe approximately 6%, 3%, and 2% better performance in each process compared to the previous results.

B. HQC

Unlike BIKE, the HQC algorithm does not have existing implementation results on a 32-bit architecture. Therefore, we used the PQClean implementation as a reference for comparison with our optimized implementation. The PQClean implementation is a standalone implementation, making it easy to port to embedded platforms. However, since it is designed for a 64-bit architecture, it is not directly suitable for our benchmarking platform. Thus, we adapted the polynomial multiplication component to fit the 32-bit environment.

Table VI shows the number of cycles required for polynomial multiplication in HQC-128, HQC-192, and HQC-256. The PQClean implementation uses the Karatsuba algorithm

Authorized licensed use limited to: DGIST. Downloaded on March 01,2026 at 08:43:07 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
PERFORMANCE COMPARISON OF POLYNOMIAL MULTIPLICATION FOR HQC-128, HQC-192, AND HQC-256

	[14]	**our code**
HQC-128	12,846,453	2,888,842
HQC-192	39,412,568	6,361,943
HQC-256	75,879,677	6,372,501

TABLE VII
PERFORMANCE COMPARISON OF HQC-128, HQC-192, AND HQC-256 IMPLEMENTATIONS

	key gen.	**encap.**	**decap.**	**imple.**
HQC-128	14,156,034	28,565,697	43,937,808	[14]
	4,201,940	8,657,454	14,075,463	**our code**
HQC-192	42,819,162	85,941,519	130,313,881	[14]
	9,771,360	19,845,917	31,170,491	**our code**
HQC-256	82,274,722	164,827,281	250,418,835	[14]
	12,757,862	25,793,534	41,868,167	**our code**

for polynomial multiplication. Compared to our optimized implementation, the performance improvement ranges from approximately 80% to up to 92% for the largest parameter set, i.e., HQC-256. Additionally, the Karatsuba-based multiplication in HQC-256 consumes about 1.9 times more cycles than in HQC-192. However, since our optimized implementation uses the FAFFT algorithm, it performs polynomial multiplication with a fixed degree of $2^{17} = 131072$, regardless of the polynomial degrees in HQC-192 and HQC-256, which are 35 851 and 57 637, respectively. As a result, the number of cycles required for polynomial multiplication in HQC-192 and HQC-256 does not differ significantly.

Table VII shows the number of cycles required to perform key generation, encapsulation, and decapsulation in HQC-128, HQC-192, and HQC-256. The performance improvements were 70% for HQC-128, 77% for HQC-192, and 84% for HQC-256.

VI. CONCLUSION

In this article, we proposed an optimization method for FAFFT to efficiently implement polynomial multiplication of BIKE and HQC on the ARM Cortex-M4. The proposed method focuses on optimizing the butterfly operation, which is the most computationally intensive part of FAFFT. We observed that the butterfly operation can be expressed as a multiplication of binary matrices, and we applied optimization techniques originally developed for linear layers in symmetric-key cryptography to accelerate this process. This leads to efficient XOR sequences for computing the matrix products. Furthermore, to effectively implement the optimized XOR sequences on the ARM Cortex-M4, we designed a register scheduling strategy that minimizes data movement between GP and FP registers.

As a result, for BIKE, our optimized FAFFT implementation reduces the cycle count for polynomial multiplication by approximately 8% and improves key generation, encapsulation, and decapsulation performance by up to 6% compared to the previous state-of-the-art implementations.

Furthermore, we present the first optimized implementation of the HQC algorithm on the ARM Cortex-M4. To evaluate our implementation, we ported the PQClean version to a 32-bit architecture and used it as a baseline. Compared to the PQClean implementation, we observed performance improvements of approximately 80% to 92% in polynomial multiplication. Additionally, our implementation improves performance in key generation, encapsulation, and decapsulation by up to 84%.

Although our implementation is optimized for the ARM Cortex-M4, the XOR sequence minimization technique itself is architecture-independent and can be applied to a wide range of processors or hardware architectures. This includes GP CPUs and even FPGA or ASIC designs, where reducing XOR complexity translates directly to improved performance and hardware efficiency.

Given that HQC was selected as a standard algorithm in March 2025, and that the ARM Cortex-M4 is the benchmark platform recommended by NIST for embedded systems, our results demonstrate the practicality of deploying HQC on resource-constrained IoT devices.

REFERENCES

[1] D. J. Bernstein et al., "Classic McEliece: Conservative code-based cryptography," presented at NIST PQC Competition, 2017.

[2] N. Aragon et al., "BIKE: Bit flipping key encapsulation," 2022. [Online]. Available: https://hal.science/hal-01671903/document

[3] C. A. Melchor et al., *Hamming Quasi-Cyclic (HQC)*, NIST, Gaithersburg, MD, USA, 2018.

[4] M. Campagna et al., "Supersingular isogeny key encapsulation," 2019. [Online]. Available: https://csrc.nist.gov/csrc/media/Projects/post-quantum-cryptography/documents/round-4/submissions/SIKE-spec.pdf

[5] R. J. McEliece, "A public-key cryptosystem based on algebraic," Nat. Aeronaut. Space Admin., Washington, DC, USA, DSN Progress Rep. 42-44, 1978.

[6] G. Alagic et al., "Status report on the fourth round of the NIST post-quantum cryptography Standardization process," NIST, Gaithersburg MD, USA, Rep. (IR) 8545, 2025.

[7] (NIST, Gaithersburg, MD, USA). *Submission Requirements and Evaluation Criteria for the Post-Quantum Cryptography Standardization Process*. (2016). [Online]. Available: https://csrc.nist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/documents/call-for-proposals-final-dec-2016.pdf

[8] G. Alagic et al., "Status report on the third round of the NIST post-quantum cryptography standardization process," NIST, Gaithersburg, MD, USA, Rep. 8413, 2022.

[9] M.-S. Chen and T. Chou, "Classic McEliece on the ARM cortex-M4," IACR, Bellevue, WA, USA, Rep. 2021/492, 2021.

[10] M.-S. Chen, T. Chou, and M. Krausz, "Optimizing BIKE for the Intel Haswell and arm cortex-M4," *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2021, no. 3, pp. 97–124, 2021.

[11] Z. Xiang, X. Zeng, D. Lin, Z. Bao, and S. Zhang, "Optimizing implementations of linear layers," IACR Trans. Symmetric Cryptol., IACR, Bellevue, WA, USA, Rep. 903/2020, 2020.

[12] J.-M. Robert and P. Véron, "Faster multiplication over F2[X] using AVX512 instruction set and VPCLMULQDQ instruction," *J. Cryptograph. Eng.*, vol. 13, no. 1, pp. 37–55, 2023.

[13] S. Deshpande, C. Xu, M. Nawan, K. Nawaz, and J. Szefer, "Fast and efficient hardware implementation of HQC," in *Proc. Int. Conf. Sel. Areas Cryptogr.*, 2023, pp. 297–321.

[14] M. J. Kannwischer, P. Schwabe, D. Stebila, and T. Wiggers, "Improving software quality in cryptography Standardization projects," in *Proc. IEEE Eur. Symp. Security Privacy*, Los Alamitos, CA, USA, 2022, pp. 19–30. [Online]. Available: https://eprint.iacr.org/2022/337

[15] "High performance, secure and low-power MCU for Zigbee®, thread™ and Bluetooth® LE 5.0 with high tx power option," Data Sheet K32W041AM/a, NXP Semiconductors, Eindhoven, The Netherlands, 2025. Accessed: Apr. 17, 2025. [Online]. Available: https://www.nxp.com/products/K32W041AM-A

Authorized licensed use limited to: DGIST. Downloaded on March 01,2026 at 08:43:07 UTC from IEEE Xplore. Restrictions apply.

[16] D. G. Cantor, "On arithmetical algorithms over finite fields," *J. Combinatorial Theory A*, vol. 50, no. 2, pp. 285–300, 1989.
[17] S.-J. Lin, W.-H. Chung, and Y. S. Han, "Novel polynomial basis and its application to reed-solomon erasure codes," in *Proc. IEEE 55th Annu. Symp. Found. Comput. Sci.*, 2014, pp. 316–325.
[18] S.-J. Lin, T. Y. Al-Naffouri, and Y. S. Han, "FFT algorithm for binary extension finite fields and its application to Reed–Solomon codes," *IEEE Trans. Inf. Theory*, vol. 62, no. 10, pp. 5343–5358, Oct. 2016.
[19] W.-D. Li, M.-S. Chen, P.-C. Kuo, C.-M. Cheng, and B.-Y. Yang, "Frobenius additive fast fourier transform," in *Proc. ACM Int. Symp. Symbolic Algebr. Comput.*, 2018, pp. 263–270.
[20] S. Banik, Y. Funabiki, and T. Isobe, "More results on shortest linear programs," in *Proc. 14th Int. Workshop Security Adv. Inf. Comput. Security*, 2019, pp. 109–128.
[21] C. Paar, "Optimized arithmetic for reed-solomon encoders," in *Proc. IEEE Int. Symp. Inf. Theory*, 1997, p. 250.
[22] L. Kölsch, "XOR-counts and lightweight multiplication with fixed elements in binary finite fields," in *Proc. 38th Annu. Int. Conf. Theory Appl. Cryptogr. Techn.*, Darmstadt, Germany, 2019, pp. 285–312.
[23] J. Boyar, P. Matthews, and R. Peralta, "Logic minimization techniques with applications to cryptology," *J. Cryptol.*, vol. 26, pp. 280–312, Apr. 2013.
[24] J. Boyar and R. Peralta, "A new combinational logic minimization technique with applications to cryptology," in *Proc. 9th Int. Symp. Exp. Algorithms*, 2010, pp. 178–189.
[25] M. J. Kannwischer, R. Petri, J. Rijneveld, P. Schwabe, and K. Stoffelen. "PQM4: Post-quantum crypto library for the ARM cortex-M4." 2019. [Online]. Available: https://github.com/mupq/pqm4

Myeonghoon Lee received the B.S. degree in mathematics from Korea University, Seoul, Republic of Korea, in 2020, where he is currently pursuing the Ph.D. degree with the Graduate School of Cyber Security.

His research focuses on post-quantum cryptography.

Jihoon Jang received the B.S. degree in mathematics and the M.S. degree in information security from Korea University, Seoul, Republic of Korea, in 2023 and 2025, respectively, where he is currently pursuing the Ph.D. degree with the Graduate School of Cyber Security.

His research interest includes post-quantum cryptography.

Suhri Kim received the B.A. degree in mathematics and the M.A. degree in information security from Korea University, Seoul, Republic of Korea, in 2014 and 2016, respectively, and the Ph.D. degree from the Graduate School of Information Security, Korea University in 2020.

She is currently an Associate Professor with the School of Mathematics, Statistics and Data Science, Sungshin Women's University, Seoul. Her research interests include post-quantum cryptography and efficient computations for isogeny-based cryptosystems.

Seokhie Hong received the M.S. and Ph.D. degrees in mathematics from Korea University, Seoul, Republic of Korea, in 1997 and 2001, respectively.

He was with SECURITY Technologies Inc. Seoul, Republic of Korea, from 2000 to 2004. Subsequently, he conducted a Postdoctoral Research with COSIC, KU Leuven, Leuven, Belgium, from 2004 to 2005, after which he joined the Graduate School of Cyber Security, Korea University. His research interests include cryptography, public and symmetric cryptosystems, hash functions, and MACs.

Authorized licensed use limited to: DGIST. Downloaded on March 01,2026 at 08:43:07 UTC from IEEE Xplore. Restrictions apply.